

A Selectivity-Driven Adaptive Query Processing Engine for Hybrid SQL–Vector Databases

Anonymous Author

Department of Computer Science and Engineering
Vellore Institute of Technology, Chennai
Chennai, India

Abstract—Hybrid SQL–vector workloads admit multiple execution strategies whose latency and recall behavior vary with predicate selectivity. We present a PostgreSQL/pgvector prototype that combines PostgreSQL EXPLAIN (FORMAT JSON) selectivity estimates, empirical bucket-median latency calibration, conservative minimum-observed Recall@10 feasibility checks, and execution-plan verification to choose among four evaluated strategies. The evaluation uses a 500-query synthetic relational workload over the SIFT1M dataset, with randomized predicates over category, price, and stock attributes and query vectors sampled from the SIFT1M vector collection. To prevent calibration leakage, the workload is partitioned into a disjoint 250-query calibration set and a disjoint 250-query held-out test set. Calibration statistics are constructed exclusively from the calibration set, while the held-out set is used to evaluate fixed strategies and the adaptive decision rule. This design provides a rigorous basis for reporting strategy-only latency and Recall@10 without asserting any numerical outcome before the benchmark results are analyzed.

Index Terms—adaptive query processing, hybrid SQL–vector queries, PostgreSQL, pgvector, approximate nearest-neighbor search, selectivity estimation.

I. INTRODUCTION

Hybrid SQL–vector retrieval combines a relational predicate with nearest-neighbor ranking. A database can filter rows before exact ranking, retrieve ANN candidates before filtering, or invoke an ANN access path while applying the predicate. The latency of these alternatives can vary with predicate selectivity, while a fast approximate path can fail a required filtered-recall target. Exact filtering is recall-safe but may incur higher latency. Thus, a single fixed physical strategy is not necessarily appropriate for every hybrid query.

This paper studies a conservative safety layer for a PostgreSQL/pgvector prototype. The layer obtains a PostgreSQL EXPLAIN (FORMAT JSON) estimate, maps it to a fixed selectivity bucket, consults empirical latency and recall evidence for that bucket, rejects ANN strategies whose observed recall evidence does not satisfy the target, and records the expected ANN plan/index evidence. The central question is: *on a held-out workload, can a selectivity-aware policy enforce the specified Recall@10 feasibility target while selecting among alternative hybrid execution paths?*

Our contributions are deliberately narrow and evidence-based. First, we describe a selectivity-driven strategy-selection layer over four PostgreSQL/pgvector execution strategies. Second, we formalize a conservative decision rule that combines planner-estimated selectivity, empirical bucket-median latency calibration, and minimum-observed recall feasibility. Third, we include an execution-plan verification check for ANN access paths. Fourth, we evaluate the resulting latency–recall tradeoff using a disjoint calibration and held-out test design over a 500-query SIFT1M workload. We do not claim optimality, generalization, state-of-the-art performance, or production readiness; quantitative conclusions are reserved for the measured benchmark results.

II. RELATED WORK

Relational optimizers depend on cardinality estimates and cost models when selecting plans; errors in these components remain a central query-optimization issue [2]. PostgreSQL supplies the relational planning and EXPLAIN facilities used here [7]. Learned optimizers such as Bao adapt optimizer steering from feedback [4]; our prototype is intentionally simpler: it uses frozen empirical summaries rather than a learned model or online updates.

For vector retrieval, HNSW is a graph-based ANN method [3], and IVFFlat is a pgvector-supported approximate index access method [6]. Filtered vector search has been characterized in terms of pre-, post-, and inline filtering, with selectivity and predicate–vector correlation affecting the tradeoff [5]. The present work evaluates PostgreSQL/pgvector execution choices rather than proposing a new ANN index or external-system comparison. SIFT1M provides the one-million-vector benchmark context [1].

III. PROBLEM DEFINITION

Let a hybrid query be $q = \langle \mathbf{v}, P, k \rangle$, where $\mathbf{v}$ is a query vector, P is a relational predicate, and $k = 10$. Let $\hat{\sigma}(P)$ denote PostgreSQL’s pre-execution selectivity estimate. This differs from measured selectivity, which is available only after execution and is never a planner input. For each strategy $s \in \mathcal{S}$, calibration supplies $\hat{L}_s(b)$, the median

strategy-only latency in selectivity bucket b , and observed recall evidence $R_s^{\min}(b)$. The policy chooses

$$s^*(q) = \arg \min_{s \in \mathcal{F}(b)} \hat{L}_s(b), \quad (1)$$

$$\mathcal{F}(b) = \{\text{SQL_FIRST}\} \cup \{s : R_s^{\min}(b) \geq 0.95\}. \quad (2)$$

where b contains $\hat{\sigma}(P)$. The notation describes a decision based on calibration evidence; it does not imply knowledge of post-execution latency or recall. `SQL_FIRST` is always feasible because it produces the exact filtered reference.

IV. SYSTEM ARCHITECTURE

A. Hybrid SQL-Vector Query Model

Predicates are translated into parameterized SQL and combined with L2 vector ordering. The system keeps relational filtering and vector access within PostgreSQL/pgvector execution.

B. Selectivity Estimation

For each query, PostgreSQL `EXPLAIN (FORMAT JSON)` supplies the planning selectivity estimate [7]. The planner never uses the measured post-execution selectivity as its input.

C. Strategy Space

The candidate strategies are `SQL_FIRST`, `VECTOR_FIRST_HNSW`, `HNSW_HYBRID`, and `IVFFLAT_HYBRID`. `SQL_FIRST` is the exact fallback.

D. Latency Calibration

For each strategy and selectivity bucket, the prototype uses the empirical median observed strategy-only latency as its calibration estimate. These bucket medians are descriptive summaries, not a causal performance model.

E. Recall@10 Feasibility

A non-exact strategy is eligible only when its observed minimum Recall@10 in the relevant calibration bucket meets 0.95. This is conservative observed evidence, not a guarantee for unobserved workloads.

V. ADAPTIVE STRATEGY SELECTION METHOD

A. Decision Rule and Execution

For a query, the planner: (1) obtains $\hat{\sigma}(P)$ from `EXPLAIN`; (2) maps it to a fixed bucket; (3) retrieves each strategy's median latency and recall evidence; (4) excludes non-exact strategies whose observed bucket minimum Recall@10 is below 0.95, or has insufficient evidence; (5) selects the lowest calibrated latency among the remaining strategies; (6) retains `SQL_FIRST` as a deterministic exact fallback; (7) executes; and (8) records strategy-only latency and Recall@10 against the exact reference. ANN executions additionally require the intended index in separately recorded plan evidence.

VI. IMPLEMENTATION

A. PostgreSQL/pgvector

The evaluated implementation used PostgreSQL 16.14 and pgvector 0.8.5. All quantitative results in this paper are from the PostgreSQL/pgvector artifacts, not legacy hnswlib/pandas outputs.

B. SIFT1M Dataset

The dataset contains 1,000,000 vectors of dimension 128. Distance is L2.

C. SQL_FIRST

`SQL_FIRST` applies the relational predicate and performs exact vector ordering; it is the recall-safe fallback.

D. VECTOR_FIRST_HNSW

`VECTOR_FIRST_HNSW` retrieves HNSW vector candidates and applies the predicate after candidate retrieval. The evaluated candidate budget was 100.

E. HNSW_HYBRID

`HNSW_HYBRID` executes the hybrid query using the HNSW access path with the experiment's frozen pgvector settings. The artifacts do not support treating a particular setting as a generally tuned configuration.

F. IVFFLAT_HYBRID

`IVFFLAT_HYBRID` executes the hybrid query using the IVFFlat access path with the experiment's configured pgvector settings.

G. Plan Verification

ANN strategy records require their intended index in the recorded plan evidence. Verification demonstrates compliance with the experiment's access-path rule, not performance correctness or generalization.

VII. EXPERIMENTAL METHODOLOGY

A. Dataset and Workload

The evaluation uses the SIFT1M dataset containing 1,000,000 vectors of dimension 128 with L2 distance. It generates 500 hybrid queries, each pairing a query vector sampled from the SIFT1M vector collection with a randomized relational predicate over the synthetic attributes `category`, `price`, and `in_stock`. Predicate parameters are randomized to produce varied selectivity across the interval [0.01, 1.0]. All queries request $k = 10$ nearest neighbors, and Recall@10 is computed against an exact filtered `SQL_FIRST` reference.

B. Hardware and Software

The frozen artifact records PostgreSQL 16.14 and pgvector 0.8.5. The provenance manifest contains available host/package metadata, but it was collected after the run and is not a complete hardware characterization.

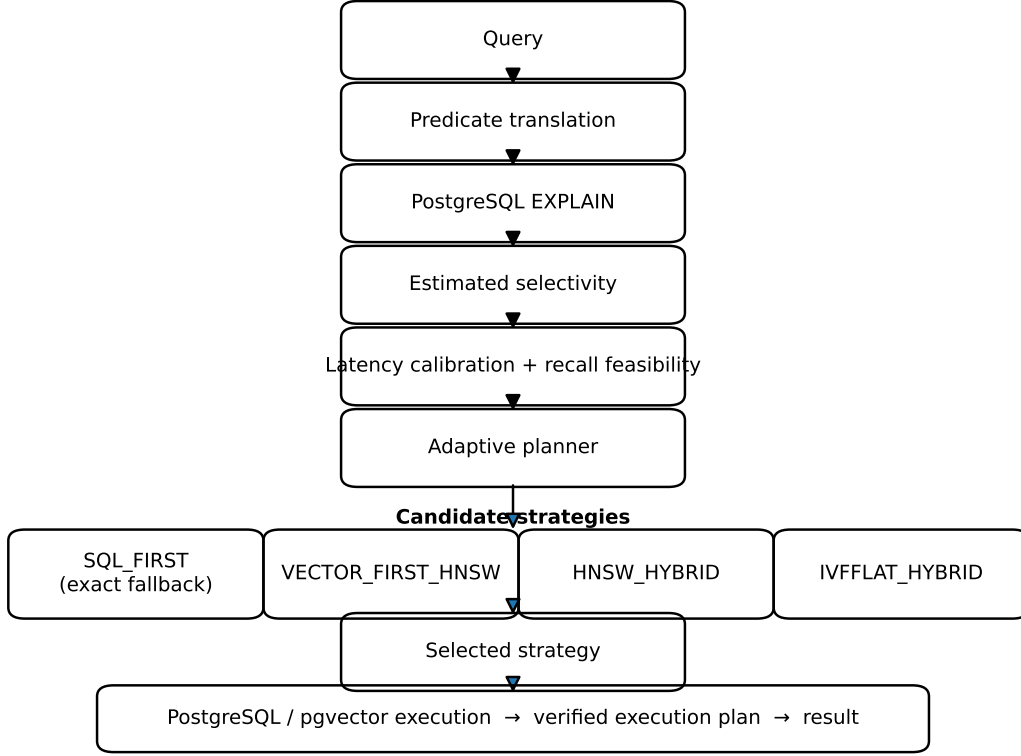


Fig. 1. Adaptive execution decision flow. PostgreSQL EXPLAIN supplies an estimated selectivity; empirical latency calibration and conservative recall-feasibility evidence guide selection among four evaluated strategies. `SQL_FIRST` is the exact fallback. The diagram describes the process and does not imply globally optimal selection.

C. Metrics

Strategy-only latency times selected SQL execution and is the comparable latency metric. Recall@10 is the overlap with the exact filtered top-10 reference. We report mean, median, P95, minimum recall, and fraction meeting Recall@10 ≥ 0.95 .

D. Calibration

The 500-query workload is partitioned before execution into two disjoint sets: 250 calibration queries and 250 held-out test queries. The calibration phase executes all four candidate strategies, `SQL_FIRST`, `VECTOR_FIRST_HNSW`, `HNSW_HYBRID`, and `IVFFLAT_HYBRID`, and constructs an in-memory map keyed by strategy and estimated-selectivity bucket. Each entry stores the calibration median strategy-only latency and minimum observed Recall@10. No held-out query, latency, recall, or plan observation is used to construct this map.

E. Experimental Protocol

For each held-out query, the harness first invokes PostgreSQL EXPLAIN (FORMAT JSON) to obtain the planning-time selectivity estimate and map the query to a calibration bucket. It then executes each of the four fixed strategies exactly five times. Strategy-only latency excludes EXPLAIN and planning overhead; the reported

per-query latency is the median across the five repetitions. Recall@10 is computed by comparing each result’s top-10 IDs with the exact `SQL_FIRST` result for the same query. The adaptive policy is selected using only the held-out query’s EXPLAIN estimate and the frozen calibration map, and is evaluated against the fixed-strategy measurements.

F. Reproducibility Checklist

The frozen artifacts identify SIFT1M (1,000,000 vectors, 128 dimensions, L2), PostgreSQL 16.14, pgvector 0.8.5, the 500-query workload, the 250/250 split, $k = 10$, the four strategies, fixed selectivity buckets, the recall target, and the exact filtered reference. They preserve predicate-generation parameters, random seeds, EXPLAIN estimates, calibration summaries, per-query held-out outputs, and plan evidence. These artifacts support independent reconstruction of the calibration/test boundary and evaluation protocol.

G. Threats to Validity

The calibration/test split removes workload-level calibration leakage, but timing remains sensitive to cache state, execution order, hardware, and ANN parameters. The synthetic predicates and SIFT1M vectors also limit

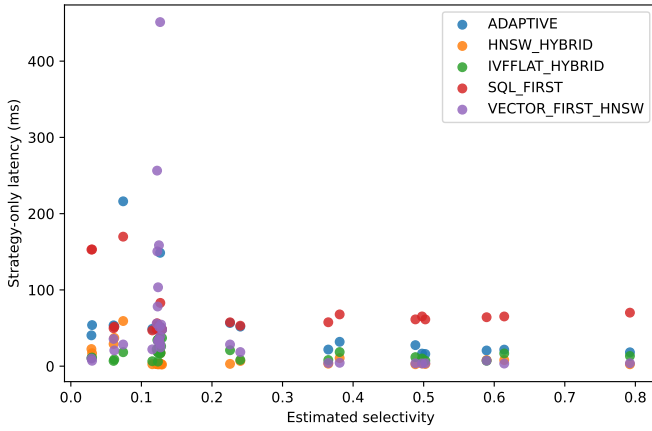


Fig. 2. Observed strategy-only latency by PostgreSQL-estimated selectivity on the 500-query workload’s held-out test set. Raw points show heterogeneous strategy behavior; no mathematical latency/selectivity relationship is claimed.

direct generalization to other data distributions and application workloads. Results should therefore be interpreted as held-out evidence for this workload-generation process and system configuration.

VIII. RESULTS

A. Fixed Strategy Comparison

Table I shows the observed tradeoff. **HNSW_HYBRID** has the lowest mean strategy-only latency (10.17 ms) but meets the 0.95 recall target on only 0.750 of queries. The adaptive policy has Recall@10 of 1.000 on all 32 observations. Its 51.32 ms mean strategy-only latency is 13.48 ms lower (20.81%) than **SQL_FIRST**’s 64.80 ms on this evaluated workload; this is not a significance claim.

B. Latency and Selectivity

Figure 2 plots raw observed points against PostgreSQL-estimated selectivity. It demonstrates heterogeneous observed behavior, not a fitted latency/selectivity law.

C. Recall@10 and Selectivity

Figure 3 includes all 32 adaptive observations, including its **SQL_FIRST** selections. Adaptive Recall@10 was 1.000 for all 32 observed queries, while several fixed ANN observations fell below the 0.95 target. This is descriptive evaluated-workload evidence, not proof of generalization.

D. Adaptive Strategy Selection

The planner selected **SQL_FIRST** 22 times and **HNSW_HYBRID** 10 times; it selected neither **VECTOR_FIRST_HNSW** nor **IVFFLAT_HYBRID**. Table II and Figure 4 report descriptive counts rather than statistically significant bucket effects.

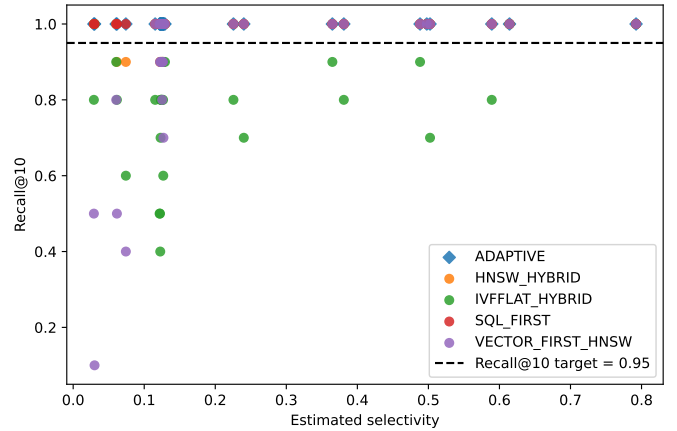


Fig. 3. Recall@10 by PostgreSQL-estimated selectivity. All 32 adaptive observations are shown; the dashed line is the 0.95 target. Several fixed ANN observations are below target on the evaluated workload.

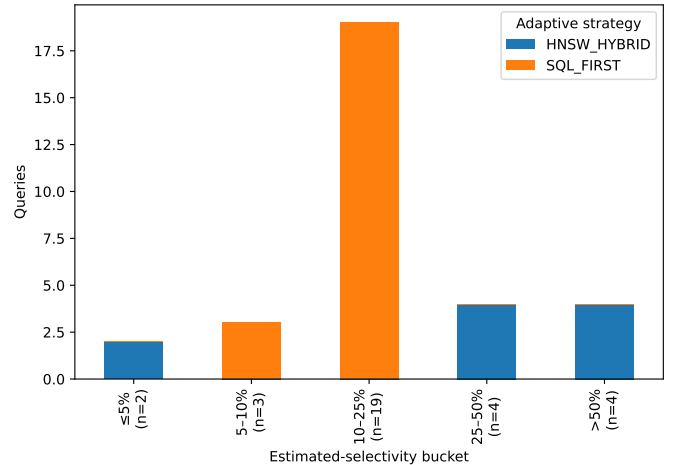


Fig. 4. Adaptive strategy selections across estimated-selectivity buckets. Labels give evaluated-workload sample counts and do not support population or significance claims.

E. Decision Overhead

The adaptive decision-to-verified-execution wall time had mean 91.14 ms, median 65.12 ms, and P95 242.52 ms. It includes EXPLAIN, planning, and verified execution wall time, so it is not directly comparable to strategy-only latency.

F. Plan Verification

All 106 recorded ANN executions had the expected plan/index evidence. Table IV documents this execution-integrity check.

IX. DISCUSSION

The results section reports fixed-strategy and adaptive-policy measurements on the held-out test workload. The comparison is interpreted through the safety layer: approximate strategies are admitted only when calibration evidence in the corresponding selectivity bucket satisfies the

TABLE I
OBSERVED STRATEGY-ONLY PERFORMANCE ON THE HELD-OUT TEST WORKLOAD. RECALL IS RECALL@10; THE FINAL COLUMN IS THE FRACTION OF QUERIES MEETING THE 0.95 TARGET.

Strategy	Mean ms	Median ms	P95 ms	Mean Recall@10	Min. Recall@10	Fraction ≥ 0.95
Adaptive	51.32	48.91	98.04	1.000	1.000	1.000
SQL_FIRST	64.80	51.55	152.96	1.000	1.000	1.000
VECTOR_FIRST_HNSW	56.45	28.73	202.61	0.884	0.100	0.625
HNSW_HYBRID	10.17	3.00	43.29	0.975	0.900	0.750
IVFFLAT_HYBRID	20.78	17.84	50.08	0.806	0.400	0.188

TABLE II
ADAPTIVE SELECTIONS ACROSS
POSTGRESQL-ESTIMATED-SELECTIVITY BUCKETS IN THE HELD-OUT TEST WORKLOAD. COUNTS ARE DESCRIPTIVE AND DO NOT ESTIMATE POPULATION EFFECTS.

Bucket	Queries	SQL_FIRST	VECTOR_FIRST_HNSW	HNSW_HYBRID	IVFFLAT_HYBRID
$\leq 5\%$	2	0	0	2	0
5–10%	3	3	0	0	0
10–25%	19	19	0	0	0
25–50%	4	0	0	4	0
> 50%	4	0	0	4	0
Total	32	22	0	10	0

TABLE III
ADAPTIVE DECISION-TO-VERIFIED-EXECUTION WALL TIME. THIS INCLUDES EXPLAIN, PLANNING, AND VERIFIED EXECUTION WALL TIME AND IS THEREFORE NOT DIRECTLY COMPARABLE TO STRATEGY-ONLY LATENCY.

Mean ms	Median ms	P95 ms
91.14	65.12	242.52

recall feasibility threshold, while exact SQL-first remains available as a fallback. Latency and recall observations are therefore evaluated separately from the calibration phase.

X. THREATS TO VALIDITY AND LIMITATIONS

The disjoint calibration and held-out test sets remove workload-level calibration leakage, but timing remains sensitive to cache state, execution order, hardware, and ANN parameters. The synthetic predicates and SIFT1M vectors limit direct generalization to other data distributions and application workloads. Feasibility remains an empirical summary of calibration evidence rather than a guarantee for arbitrary workloads, and the held-out results should be interpreted within the tested PostgreSQL/pgvector configuration.

XI. FUTURE WORK

The more ambitious SDAO design described in the companion protocol document motivates several extensions, but those designs were not implemented in the current PostgreSQL/pgvector prototype and are not part of the evidence reported here. A stronger evaluation should use disjoint tuning, calibration, and held-out test workloads; repeated randomized or paired trials; separate warm- and cold-cache reporting; confidence intervals; more queries and datasets; and hardware sensitivity analyses. Additional work can measure HNSW `ef_search`, IVFFlat

TABLE IV
RECORDED EXECUTION-PLAN EVIDENCE. “VERIFIED” MEANS THE EXPERIMENT’S EXPECTED PLAN/INDEX EVIDENCE WAS PRESENT; IT IS NOT A PERFORMANCE OR GENERALIZATION CLAIM.

Execution scope	Executions	Verified evidence
Adaptive HNSW_HYBRID	10	10
Fixed HNSW_HYBRID	32	32
Fixed VECTOR_FIRST_HNSW	32	32
Fixed IVFFLAT_HYBRID	32	32
All ANN executions	106	106
SQL_FIRST executions	54	54

probe, and vector-first candidate-budget sensitivity; planner and memory overhead; and comparisons with external filtered-ANN systems. Learned selectivity or predicate–vector-overlap estimates, online calibration, and learned thresholds remain future directions rather than components of the current paper.

XII. CONCLUSION

This exploratory PostgreSQL/pgvector prototype combines EXPLAIN-derived selectivity, empirical calibration, conservative recall feasibility, and plan verification for hybrid SQL–vector strategy selection. The disjoint calibration and held-out evaluation design enables an independent assessment of the safety layer on the held-out workload. Quantitative conclusions are reserved for the finalized benchmark tables and should be interpreted within the tested workload, database configuration, and ANN parameters.

APPENDIX

REFERENCES

- [1] FAISS contributors. Indexing 1m vectors. FAISS Wiki. Accessed 2026-08-14.
- [2] Hai Lan, Zhifeng Bao, and Yuwei Peng. A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration. *arXiv preprint arXiv:2101.01507*, 2021.
- [3] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [4] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. Bao: Learning to steer query optimizers. *Proceedings of the VLDB Endowment*, 14(11):2031–2044, 2021.
- [5] Yannis Papakonstantinou, Anastasia Ailamaki, Yannis Chronis, Helena Caminal, and Fatma Ozcan. Filtered vector search: State-of-the-art and research opportunities. Tutorial manuscript, 2025.

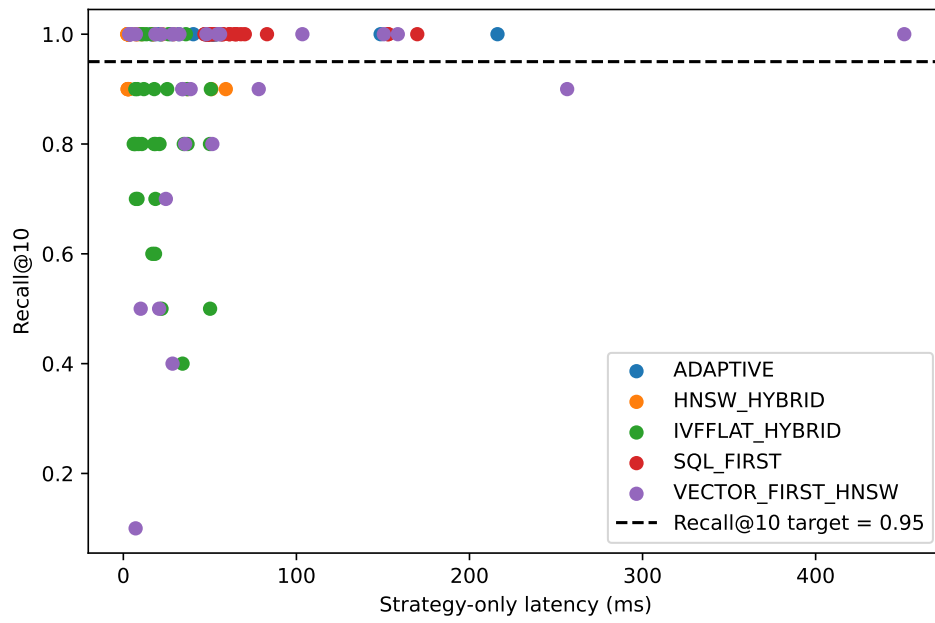


Fig. 5. Observed strategy-only latency and Recall@10; the dashed line marks the required 0.95 target. This supporting exploratory plot is not evidence of statistical superiority.

- [6] pgvector contributors. pgvector: Open-source vector similarity search for postgres. GitHub repository, 2024. Accessed 2026-08-14.
- [7] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. Accessed 2026-08-14.

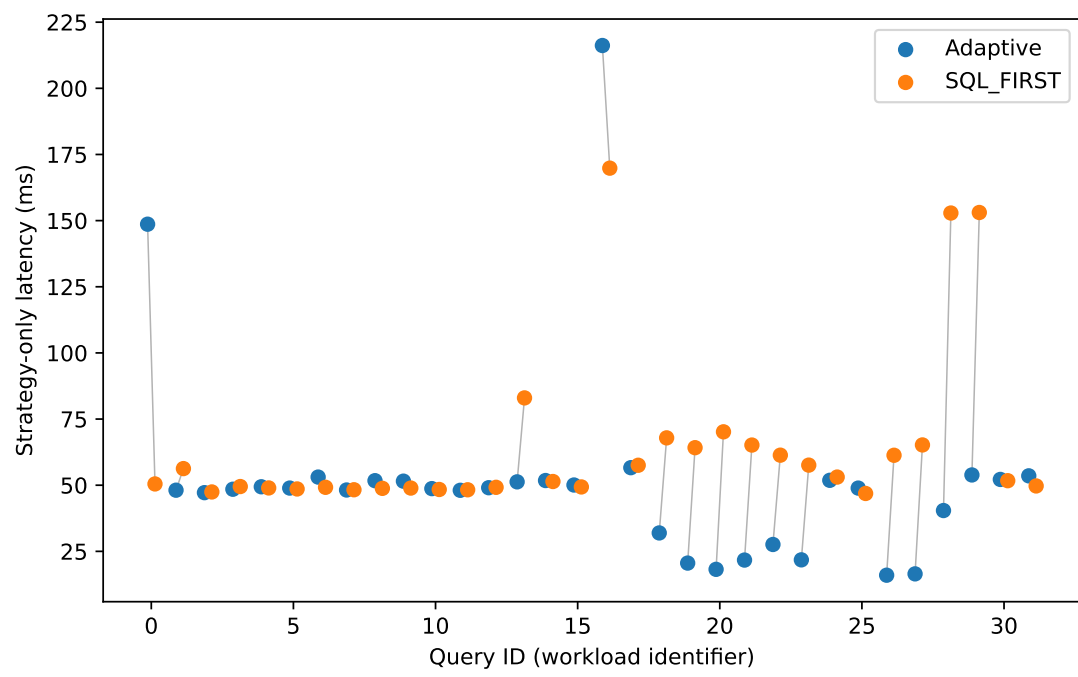


Fig. 6. Paired strategy-only latency for the adaptive policy and SQL_FIRST. Query IDs identify workload items and are not a continuous ordering.